Práctica Integradora – Emprendimientos, Responsabilidad y Organización en IT

Tecnicatura Universitaria en Programación a Distancia – UTN

Alumno: Ignacio Figueroa · **DNI:** 45.406.120 · **Fecha:** Oct 26, 2026

Introducción

El caso trata de tres desarrolladores que arman una app de turnos médicos sin ningún tipo de acuerdo formal. Mientras era un proyecto chico funcionaba, pero cuando aparecieron usuarios, plata y una falla que le hizo perder dinero a un cliente, quedó claro que nadie sabía quién era dueño de qué ni quién tenía que hacerse cargo.

En este trabajo voy a analizar el caso siguiendo las actividades de la consigna y al final propongo cómo lo organizaría yo. Lo que más me interesó del caso es que muestra algo que como programadores a veces no pensamos: que el código puede estar bien y aun así el proyecto fracasar por no haber resuelto la parte organizativa a tiempo.

Actividad 1 – Análisis del proyecto

1. ¿En qué etapa se encuentra el proyecto: informal u organizada?

El proyecto está en una etapa informal, aunque ya no debería estarlo. No hay estructura legal, no hay contratos entre los tres y todo se decide de palabra. El problema es que el proyecto creció como si fuera una empresa (tiene usuarios, clientes y facturación) pero sigue funcionando como un grupo de amigos programando juntos. Yo diría que está en una etapa de transición que nunca se terminó de hacer: por lo que genera ya necesita una organización formal, pero por cómo funciona sigue siendo informal.

2. ¿Qué cambios indican que el proyecto dejó de ser solo técnico?

Hay varias señales claras:

- Aparecen **usuarios reales** que dependen del sistema para sacar turnos médicos, o sea que una falla ya no afecta solo a los desarrolladores.
- Empiezan a entrar **ingresos**, y con eso surge la pregunta de cómo se reparte la plata y quién factura.

- Hay **clientes (empresas)** con las que existe algún tipo de relación comercial, aunque no esté escrita.
- Surgen **nuevas responsabilidades**: soporte, mantenimiento, disponibilidad del servicio y, por ser turnos médicos, manejo de datos personales de pacientes.
- Cuando el sistema falla aparecen **conflictos entre las personas**, no solo errores en el código.

Cuando el proyecto empieza a involucrar a terceros, dinero y obligaciones, deja de ser solamente un tema de programación y pasa a ser también un tema legal, económico y de gestión.

3. ¿Por qué el crecimiento del proyecto exige una nueva forma de organización?

Porque a mayor tamaño, más cosas pueden salir mal y más gente se ve afectada. Al principio, si algo fallaba, lo arreglaban entre ellos y listo. Ahora una falla le genera pérdidas a una empresa, y esa empresa va a querer saber a quién reclamarle.

Los acuerdos verbales sirven mientras todos estén de acuerdo, pero cuando aparece un conflicto cada uno recuerda las cosas a su manera y no hay nada para demostrar qué se había dicho. Además, un proyecto con ingresos necesita poder facturar, firmar contratos con clientes y tener reglas claras para repartir ganancias. Nada de eso se puede hacer bien sin una estructura formal. Por eso el crecimiento no es solo una buena noticia: también obliga a ordenarse.

Actividad 2 – Organización y sociedades

1. ¿Qué problemas genera la falta de formalización?

En el caso se ven casi todos los problemas que puede traer no formalizar:

- **Propiedad del software sin definir:** si mañana uno de los tres se va, no está claro si puede llevarse el código, si puede usarlo para otro proyecto o si los otros dos pueden seguir sin él.
- **Reparto de ganancias confuso:** sin un acuerdo, cada uno puede sentir que aportó más que el resto, y eso termina en peleas.
- **Nadie tiene la última palabra:** si los tres opinan distinto, no hay forma de destrabar una decisión.
- **Responsabilidad difusa ante errores:** cuando el sistema falla, todos se pasan la culpa.
- **Riesgo patrimonial personal:** al no haber una sociedad constituida, un reclamo puede terminar afectando los bienes personales de cada uno.

- **Poca seriedad frente a clientes:** a una empresa le cuesta más confiar en un servicio que no tiene contratos ni una figura legal detrás.

Esto se relaciona con la diferencia entre organización formal e informal que vimos en la unidad. La organización informal se apoya en la confianza y en la costumbre; la formal se apoya en reglas escritas, roles definidos y una estructura reconocida legalmente. La informal no es mala en sí, de hecho es normal al arrancar, pero tiene un límite, y este proyecto lo pasó.

2. ¿Qué tipo de organización sería recomendable para este proyecto?

Para mí lo más recomendable es formar una **sociedad comercial**. En Argentina, para un emprendimiento chico de tres socios, las opciones más lógicas serían una **SAS (Sociedad por Acciones Simplificada)** o una **SRL (Sociedad de Responsabilidad Limitada)**.

Yo me inclinaría por la **SAS**, porque se puede constituir más rápido, con menos costos y trámites más simples, y está pensada justamente para emprendimientos. Además, si en el futuro quieren sumar un inversor, el esquema por acciones lo facilita. La SRL también serviría, sobre todo si quieren algo más cerrado entre los tres.

Seguir como tres freelancers separados no lo veo viable, porque el producto es uno solo, es compartido y tiene clientes en común.

3. ¿Qué ventajas aporta la creación de una sociedad?

- **Responsabilidad limitada:** los socios responden en principio hasta el capital que aportaron, y no con todo su patrimonio personal.
- **El software pasa a ser de la sociedad:** se puede ceder el código a la empresa y así queda claro quién es el dueño, aunque un socio se vaya.
- **Reglas escritas:** el estatuto y un acuerdo de socios definen cómo se decide, cómo se reparten las ganancias y qué pasa si alguien quiere salir.
- **Capacidad para contratar y facturar** como empresa, con más credibilidad frente a clientes.
- **Continuidad:** la sociedad sigue existiendo aunque cambien las personas.

4. ¿Cómo cambia la responsabilidad al pasar de freelance a sociedad?

Como freelance (por ejemplo, un monotributista), la persona responde con **su propio patrimonio** por lo que hace. Si comete un error y genera un daño, el reclamo va directo contra ella. Y si tres personas trabajan juntas sin formalizar, la ley las puede considerar una sociedad no constituida regularmente (lo que antes se llamaba "sociedad de hecho"), donde los socios también responden con sus bienes personales.

En una sociedad como una SAS o una SRL, la que firma los contratos y responde frente al cliente es **la sociedad como persona jurídica**, separada de los socios. El riesgo de cada socio queda en general limitado a su aporte. Hay que aclarar que esto no es un escudo total: si un administrador actúa con dolo o con una negligencia grave, puede responder personalmente. Pero el cambio es muy importante, porque separa el riesgo del negocio del patrimonio personal de cada uno.

Actividad 3 – Responsabilidad en el proyecto

1. ¿Existe responsabilidad por las pérdidas generadas?

Sí, creo que existe. Que no haya un contrato escrito no significa que no haya obligaciones. Si los desarrolladores le ofrecieron un servicio a una empresa y esa empresa lo usaba (y seguramente pagaba por él), hay una relación de hecho. Si el sistema falló y eso le causó un daño económico, la empresa puede reclamar. Lo que sí va a depender de cada caso es cuánto se puede reclamar, ya que hay que demostrar el daño, que la falla lo causó y que hubo algún tipo de culpa o incumplimiento.

2. ¿Qué tipo de responsabilidad podría aplicarse?

Según lo visto en la unidad, se podrían dar varios tipos:

- **Responsabilidad civil contractual:** si existía un acuerdo con la empresa cliente (aunque fuera verbal o por mail), el sistema no funcionó como se había pactado y eso es un incumplimiento.
- **Responsabilidad civil extracontractual:** si no se puede probar un acuerdo, igual existe el deber general de no dañar a otro. Si el daño fue por negligencia (por ejemplo, subir a producción sin testear), se puede reclamar por esta vía.
- **Responsabilidad profesional:** como desarrolladores se espera que trabajen con un nivel razonable de cuidado: hacer pruebas, tener backups, controlar los cambios antes de publicarlos.
- **Responsabilidad por datos personales:** al ser una app de turnos médicos, maneja datos de salud, que la Ley 25.326 considera datos sensibles. Si la falla expuso o perdió esos datos, la cosa se agrava bastante.

La responsabilidad penal la veo poco probable, porque haría falta intención de causar el daño y no parece ser el caso.

3. ¿Quién debería responder en este caso?

Esta es justamente la pregunta más difícil del caso, porque al no haber nada definido la respuesta no es clara. Como no hay una sociedad constituida, frente al cliente terminarían respondiendo **los tres desarrolladores**, porque actuaron juntos ofreciendo el mismo servicio. El cliente no tiene por qué saber cuál de los tres escribió el código que falló.

Después, puertas adentro, se podría discutir si alguno tuvo más culpa que otro (por ejemplo, el que hizo el deploy sin probar). Pero eso es un problema interno entre ellos y no le cambia nada al cliente. En mi opinión lo justo es que respondan los tres, y que entre ellos se repartan el costo según lo que acuerden.

4. ¿Cómo influye la falta de acuerdos en la determinación de responsabilidad?

Influye muchísimo. Sin acuerdos:

- no hay un contrato con el cliente que limite la responsabilidad o defina qué nivel de servicio se prometía;
- no hay roles definidos, entonces no se sabe quién estaba a cargo de testear, de controlar producción o de atender al cliente;
- no hay una sociedad que absorba el reclamo, así que el riesgo cae sobre las personas;
- no hay pruebas de lo que se habló, entonces todo es la palabra de uno contra la del otro.

En la unidad vimos la pregunta "¿quién responde?" y la idea de que la responsabilidad se determina mejor cuando están claros los roles y las obligaciones de cada uno. Este caso es el ejemplo de lo contrario: como nada estaba definido, todos quedan expuestos y nadie puede defenderse bien.

Actividad 4 – Decisiones y gestión del proyecto

1. ¿Qué decisiones no se tomaron correctamente?

Más que decisiones mal tomadas, el problema es que muchas decisiones directamente **no se tomaron**. Las principales:

- No definir quién es dueño del software.
- No acordar cómo repartir las ganancias.
- No establecer quién decide y cómo se resuelven los desacuerdos.
- No pensar qué pasa si el sistema falla (ni con el cliente ni entre ellos).
- No firmar contratos con los clientes.
- No formalizar el proyecto cuando empezaron a entrar ingresos.

Y del lado técnico, que también es una decisión de gestión: aparentemente no había un proceso claro de pruebas y control antes de pasar cambios a producción.

2. ¿En qué momento deberían haberse tomado?

No todo hacía falta resolverlo el primer día, pero cada tema tenía su momento:

1. **Al arrancar:** un acuerdo básico entre los tres, aunque sea simple y por escrito: quién aporta qué, de quién es el código y cómo se decide.
2. **Al conseguir los primeros usuarios:** términos y condiciones de uso y una política de privacidad, sobre todo por manejar datos de salud.
3. **Al empezar a cobrar:** constituir la sociedad, definir el reparto de ingresos y firmar contratos con los clientes.
4. **Antes de tener clientes empresa en producción:** procesos de testing, backups, monitoreo y un plan para responder ante incidentes.

El error fue ir postergando todo con la idea de "ya lo vemos más adelante", hasta que el problema apareció solo.

3. ¿Por qué se dice que el problema no es técnico sino organizativo?

Porque las fallas en un sistema siempre pueden pasar; ningún software está libre de bugs. Lo que convirtió una falla técnica en una crisis fue todo lo que no estaba organizado alrededor: nadie sabía quién tenía que responder, no había contrato con el cliente, no había reglas para resolver el conflicto entre los socios y no había un plan de contingencia.

Si el proyecto hubiera estado bien organizado, la misma falla se habría manejado como un incidente más: se arregla, se le responde al cliente según lo pactado y la sociedad asume el costo. En cambio, acá la falla destapó todos los problemas que ya estaban y nadie había querido enfrentar. El bug fue el disparador, no la causa.

4. ¿Cómo impactan estas decisiones en el futuro del proyecto?

Si no hacen nada, el futuro del proyecto es bastante complicado:

- Los conflictos internos pueden terminar con alguno de los socios yéndose, y sin saber de quién es el código puede volverse imposible seguir.
- El cliente afectado puede reclamar judicialmente y comprometer el patrimonio de cada uno.
- Otros clientes pueden perder la confianza y dejar de usar la app.
- Sin una estructura formal es difícil crecer, conseguir inversión o contratar gente.

En cambio, si usan esta crisis para ordenarse, puede ser un punto de inflexión positivo. Las decisiones que se tomen ahora son las que van a definir si el proyecto se convierte en una empresa o si se disuelve. Esto conecta con lo que vimos en la unidad sobre el impacto de las decisiones: lo que se decide (o no se decide) al principio condiciona todo el crecimiento posterior.

Actividad integradora final – Propuesta de solución

Mi propuesta es convertir el proyecto en una SAS con tres socios iguales, roles definidos por escrito y un acuerdo de socios que cubra todo lo que hoy está sin resolver. A continuación detallo cada punto.

Estructura organizativa

Propongo constituir una **SAS** con los tres desarrolladores como socios, con un tercio de las acciones cada uno. Elijo la SAS por la responsabilidad limitada, porque es más rápida y barata de armar que otros tipos societarios y porque deja abierta la posibilidad de sumar inversores más adelante.

Al constituirla, cada uno debería **ceder a la sociedad los derechos sobre el código** que escribió hasta ahora, así el software queda como propiedad de la empresa y no de una persona. Además del estatuto, firmarían un **acuerdo de socios** con las reglas internas que se detallan abajo.

Roles de cada integrante

Como la consigna no dice qué hace cada uno, los asigno de forma tentativa según tareas típicas de un proyecto así:

Integrante	Rol	Responsabilidades principales
Desarrollador A	Administrador de la SAS / Líder de producto	Representa a la sociedad, firma contratos, atiende a los clientes y prioriza qué se desarrolla
Desarrollador B	Líder técnico (backend e infraestructura)	Arquitectura, base de datos, servidores, backups, seguridad y despliegues a producción

Integrante	Rol	Responsabilidades principales
Desarrollador C	Líder de calidad y frontend	Interfaz de la app, testing, control de calidad antes de cada versión y soporte a usuarios

Los roles no significan que cada uno trabaje solo en lo suyo; los tres siguen programando, pero cada área tiene un responsable claro que rinde cuentas.

Criterios de toma de decisiones

Propongo tres niveles:

1. **Decisiones del día a día** (qué tarea hacer, cómo resolver un bug, detalles técnicos): las toma el responsable del área.
2. **Decisiones importantes** (nuevos clientes, precios, contratar a alguien, gastos grandes, cambios de tecnología): por **mayoría**, o sea dos de tres socios.
3. **Decisiones críticas** (vender la empresa, sumar un socio o inversor, cambiar el reparto, disolver la sociedad): por **unanimidad**.

Todo lo importante se decide en una reunión mensual y queda en un acta o documento compartido, para que nunca más dependa de la memoria de cada uno.

Distribución de ingresos

- Cada socio cobra un **monto fijo mensual** por su trabajo en la empresa, igual para los tres mientras la dedicación sea parecida.
- De lo que sobra, un **20% se reserva** como fondo para imprevistos (por ejemplo, un reclamo como el del caso), servidores y mejoras.
- El resto de la ganancia se reparte **según las acciones**, un tercio para cada uno.
- Si en algún momento la dedicación cambia mucho (por ejemplo, uno pasa a trabajar medio tiempo), el monto fijo se ajusta con el acuerdo de los tres.

Medidas para reducir riesgos

Legales y comerciales

- Firmar **contratos con todos los clientes**, con el alcance del servicio, el nivel de disponibilidad comprometido y un límite razonable de responsabilidad ante fallas.
- Publicar **términos y condiciones** y una **política de privacidad** en la app.

- Cumplir con la **Ley 25.326 de protección de datos personales**, ya que los datos de salud son sensibles: acceso restringido, cifrado y registro de las bases de datos.
- Evaluar un **seguro de responsabilidad profesional** cuando el volumen de clientes lo justifique.

Técnicas

- Tener un **entorno de pruebas** separado de producción y no subir nada sin probar.
- Hacer **revisión de código** entre los socios antes de cada cambio importante.
- **Backups automáticos** y probados cada cierto tiempo.
- **Monitoreo** del sistema con alertas para detectar fallas rápido.
- Un **plan de respuesta a incidentes**: quién se encarga, cómo se avisa al cliente y en cuánto tiempo.

Acciones para evitar conflictos futuros

- Dejar **todo por escrito**: acuerdo de socios, actas de reuniones y contratos.
- Incluir en el acuerdo una **cláusula de salida**: si un socio se quiere ir, los otros tienen prioridad para comprarle sus acciones a un valor calculado de una forma acordada de antemano.
- Definir qué pasa con un socio que deja de aportar trabajo (por ejemplo, que vaya perdiendo parte de su participación si se va antes de cierto tiempo).
- Hacer una **reunión mensual** para revisar números, avances y problemas, en lugar de esperar a que las cosas exploten.
- Pactar que, ante un desacuerdo que no se pueda resolver entre ellos, primero se recurra a una **mediación** antes de ir a juicio.
- Hacer un **análisis post-incidente** después de cada falla, enfocado en qué mejorar en el proceso y no en buscar culpables.

Qué harían ahora con el problema del cliente

Antes de todo lo anterior, hay que resolver la crisis actual. Lo que yo haría: arreglar la falla y documentar qué pasó, comunicarse con la empresa cliente de forma transparente, intentar llegar a un acuerdo (por ejemplo, meses de servicio sin cargo o una compensación) y que el costo lo asuman los tres en partes iguales, ya que en ese momento no había reglas que dijeran otra cosa.

Conclusión

Haciendo este trabajo me di cuenta de que un proyecto de software no se sostiene solo con buen código. Los tres desarrolladores del caso probablemente eran buenos programadores, y la app funcionó lo suficiente como para conseguir usuarios y clientes. Pero no pensaron en la parte organizativa, legal y de responsabilidades, y eso fue lo que los dejó expuestos cuando apareció el primer problema serio.

Lo que más me quedó de la unidad es que formalizar no es burocracia innecesaria, sino una forma de protegerse a uno mismo, a los socios y a los clientes. Y que las decisiones importantes conviene tomarlas cuando todos se llevan bien, no cuando ya hay un conflicto. Como futuro programador, creo que es algo que tengo que tener en cuenta si alguna vez arranco un emprendimiento propio con otras personas: antes de escribir la primera línea de código, tener por lo menos un acuerdo claro sobre cómo vamos a trabajar.